

CO5 ASSESSMENT 1

CONSTRAINT-BASED PROBLEM SOLVING

1. Real-Time Face Recognition for Campus Security

1. Problem Statement

A university requires a real-time security system that captures faces from CCTV cameras, detects faces under different lighting conditions, extracts facial features, and identifies registered students and staff accurately while running on a low-cost computer.

2. Objective

To design an OpenCV-based face recognition pipeline that provides accurate and fast identification with minimum false recognition.

3. Constraints

- Real-time video processing.
- Different lighting conditions.
- Multiple faces in a frame.
- Limited computational resources.
- Minimum false recognition.
- Temporary face angle and movement variations.

4. Input

- Live CCTV/video stream.
- Registered face images of students and staff.
- Person ID/name database.

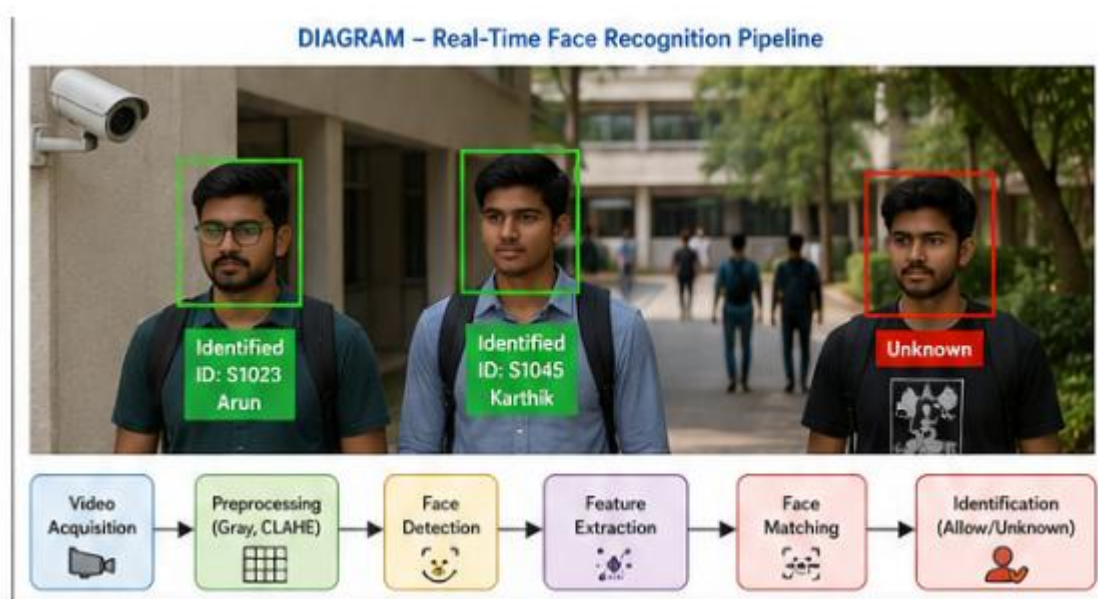
5. Processing / Algorithm

1. Capture frames from the CCTV camera using OpenCV.
2. Resize the frame to reduce computational cost.
3. Convert the frame to grayscale.
4. Improve illumination using histogram equalization or CLAHE.

5. Detect faces using Haar Cascade or OpenCV DNN face detector.
6. Crop the detected face region.
7. Normalize the face size and alignment.
8. Extract facial features using LBPH or a face embedding model.
9. Compare the extracted features with registered face features.
10. Apply a confidence/distance threshold to reduce false recognition.
11. Display the recognized person's name and confidence on the video.
12. Store unknown faces as “Unknown”.
13. Process selected frames or use a smaller frame resolution to maintain real-time performance.

6. OpenCV Pipeline

Figure: OpenCV-based system diagram



Camera → Frame Acquisition → Resize → Grayscale/CLAHE → Face Detection → Face Alignment → Feature Extraction → Feature Matching → Threshold Check → Identification → Display

7. Output

- Detected face bounding boxes.
- Student/staff name or ID.
- “Unknown” for unregistered persons.
- Real-time security display.

8. Justification

Grayscale conversion reduces computation, while CLAHE improves performance under uneven lighting. Face detection locates only relevant regions, reducing processing time. LBPH is suitable for low-cost systems because it is computationally lightweight. A confidence threshold prevents uncertain matches from being accepted. Frame resizing and processing selected frames improve real-time performance.

2. Intelligent Traffic Violation Detection System

1. Problem Statement

A traffic department wants to automatically detect, track, count, and analyze vehicles from roadside CCTV cameras and generate real-time alerts for vehicles crossing predefined lines or showing abnormal movement.

2. Objective

To develop an OpenCV-based traffic monitoring system that accurately detects and tracks vehicles while minimizing counting errors.

3. Constraints

- Multiple vehicles may appear simultaneously.
- Vehicles may overlap.
- Shadows can be detected as objects.
- Camera noise and illumination changes.
- Dynamic backgrounds.
- Vehicles must be tracked across frames.
- Real-time processing is required.

4. Input

- Live CCTV/video stream.
- Predefined virtual counting line.
- Region of interest.
- Vehicle detection parameters.

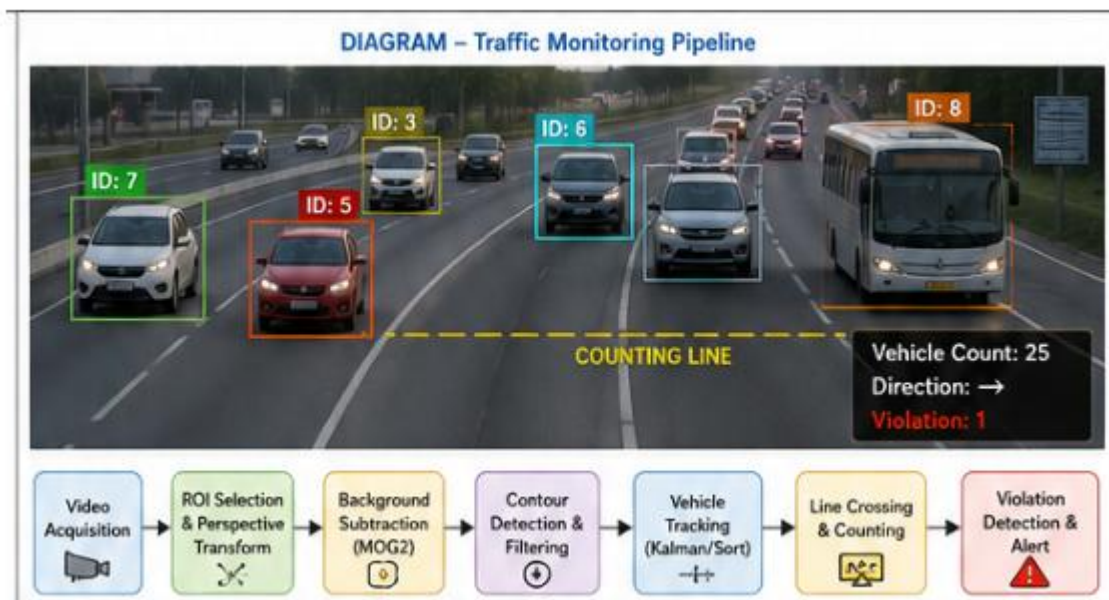
5. Processing / Algorithm

1. Capture video frames using OpenCV.
2. Resize frames for faster processing.
3. Define the road region of interest.
4. Convert frames into a suitable color space or grayscale.
5. Apply background subtraction such as MOG2 or KNN.
6. Remove noise using Gaussian/median filtering.

7. Apply morphological opening to remove small noise.
8. Apply morphological closing to fill gaps in vehicle regions.
9. Detect contours or use an object detector to identify vehicles.
10. Calculate the centroid of each detected vehicle.
11. Assign an ID to each vehicle.
12. Track vehicles using centroid tracking, Kalman filtering, or another tracking method.
13. Check whether the vehicle centroid crosses the predefined line.
14. Increment the vehicle count only once for each vehicle.
15. Analyze direction and unusual movements.
16. Display vehicle IDs, count, movement direction, and violation alerts.

6. OpenCV Pipeline

Figure: OpenCV-based system diagram



CCTV → Frame Acquisition → ROI Selection → Preprocessing → Background Subtraction/Object Detection → Morphological Filtering → Contour Detection → Vehicle Tracking → Line Crossing Detection → Violation Detection → Visualization

7. Output

- Real-time vehicle count.

- Vehicle tracking IDs.
- Direction of movement.
- Line-crossing/violation alerts.
- Bounding boxes around vehicles.

8. Handling Errors

- Shadows: Use shadow detection in MOG2 and morphological processing.
- Camera noise: Apply Gaussian or median filtering.
- Dynamic background: Use adaptive background subtraction.
- Multiple vehicles: Assign unique tracking IDs.
- Overlapping vehicles: Maintain tracking using position and motion information.
- Duplicate counting: Count only when a tracked vehicle crosses the line for the first time.

9. Justification

ROI selection reduces unnecessary processing. Background subtraction detects moving vehicles efficiently, while morphological operations remove small noise. Tracking maintains vehicle identity between frames. A virtual line and unique vehicle ID ensure that the same vehicle is counted only once.

3. Automated Document Processing System

1. Problem Statement

A company receives thousands of scanned documents containing blur, noise, uneven illumination, different font sizes, and unwanted backgrounds. The system must automatically enhance these documents and prepare them for accurate OCR.

2. Objective

To design an OpenCV image-processing pipeline that improves document quality and extracts readable text efficiently using OCR.

3. Constraints

- Blurred documents.
- Uneven illumination.
- Background noise.
- Different font sizes.
- Skewed documents.
- Large number of documents.
- Batch processing requirement.

4. Input

- Scanned document images.
- Images in JPG, PNG, TIFF, or similar formats.

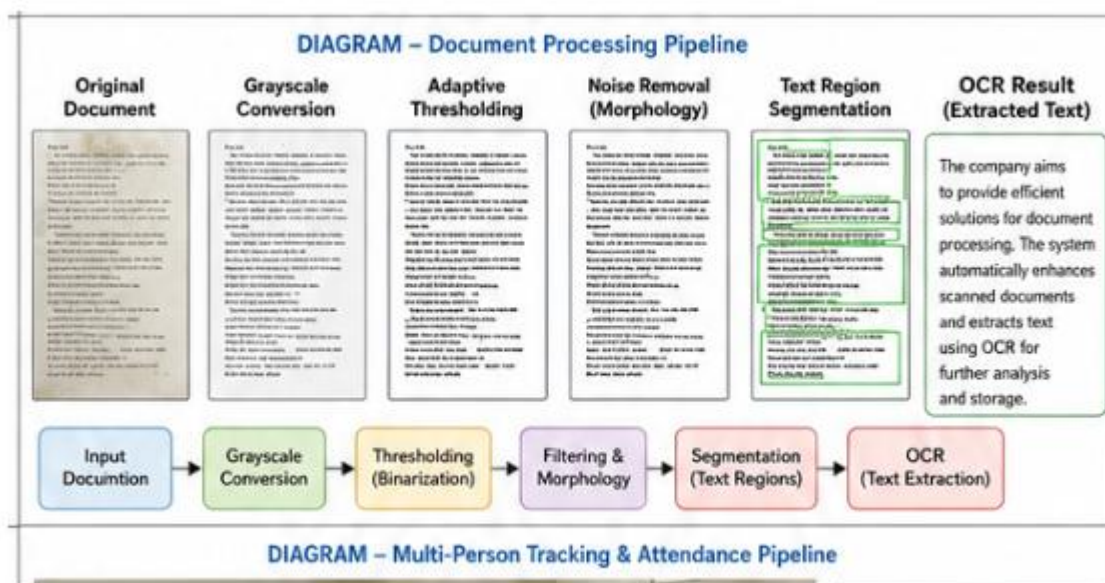
5. Processing / Algorithm

1. Read the document using OpenCV.
2. Resize the image if required.
3. Convert the image to grayscale.
4. Apply Gaussian or median filtering to reduce noise.
5. Correct uneven illumination using adaptive techniques.
6. Apply thresholding.
7. Use adaptive thresholding or Otsu thresholding to separate text from background.
8. Apply morphological opening to remove small unwanted objects.

9. Apply morphological closing to connect broken characters.
10. Detect document boundaries using contours.
11. Apply perspective transformation to correct document geometry.
12. Segment text regions or lines.
13. Prepare the enhanced image for OCR.
14. Pass the processed image to an OCR engine.
15. Store extracted text in a file/database.
16. Repeat the process for all documents in batch mode.

6. OpenCV Pipeline

Figure: OpenCV-based system diagram



Input Document → Grayscale → Noise Filtering → Illumination Correction → Thresholding → Morphological Operations → Segmentation → Contour Detection → Perspective Correction → OCR → Text Storage

7. Output

- Enhanced document image.
- Segmented text regions.
- Extracted machine-readable text.
- Processed document files.

8. Role of Each Stage

- Grayscale: Reduces a color image to one intensity channel and decreases computation.
- Filtering: Removes noise and improves character clarity.
- Thresholding: Separates foreground text from the background.
- Morphological operations: Remove small noise and repair broken text.
- Segmentation: Separates text lines/regions for better OCR.
- Geometric correction: Straightens skewed or perspective-distorted documents.
- OCR: Converts processed characters into editable text.

9. Justification

Preprocessing improves the contrast and readability of characters before OCR. Adaptive thresholding handles uneven illumination better than a single global threshold. Morphological operations remove unwanted noise and connect broken characters. Perspective correction produces a properly aligned document. Batch processing allows thousands of documents to be processed automatically.

4. Real-Time Multi-Person Tracking and Attendance System

1. Problem Statement

A college wants to automatically record student attendance using classroom CCTV. The system must detect multiple students, track them across frames, recognize registered students, and prevent duplicate attendance.

2. Objective

To design an OpenCV-based face recognition and tracking system that records each student's attendance once while operating continuously in real time.

3. Constraints

- Multiple students in one frame.
- Head movement and different face orientations.
- Temporary face disappearance.
- Duplicate attendance must be avoided.
- Real-time processing.
- Limited computational resources.

4. Input

- Classroom CCTV video.
- Registered student face database.
- Student ID and name information.

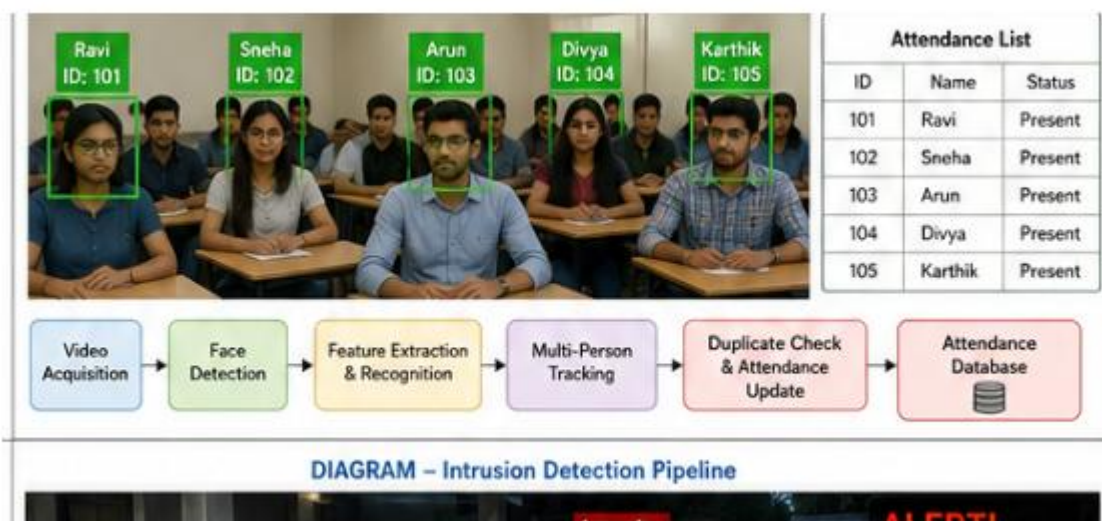
5. Processing / Algorithm

1. Capture frames from the classroom CCTV.
2. Resize frames for faster processing.
3. Detect all visible faces.
4. Convert detected face regions to grayscale.
5. Normalize and align each face.
6. Extract facial features using LBPH or face embeddings.
7. Compare features with the registered student database.
8. Assign an identity to each recognized student.

9. Create a unique tracking ID for every detected person.
10. Track each person across consecutive frames.
11. If a face temporarily disappears, maintain its identity for a limited number of frames.
12. Confirm recognition using multiple consecutive frames.
13. Check the attendance database before marking attendance.
14. Mark a student present only if the student is not already recorded.
15. Display bounding boxes, names, and attendance status.
16. Continue processing until the CCTV session ends.

6. OpenCV Pipeline

Figure: OpenCV-based system diagram



CCTV → Frame Acquisition → Face Detection → Preprocessing → Feature Extraction → Face Recognition → Multi-Person Tracking → Identity Association → Attendance Verification → Attendance Database

7. Output

- Detected student faces.
- Student names/IDs.
- Tracking IDs.
- Attendance status.

- Final attendance list.

8. Preventing Duplicate Attendance

If recognized student ID is not in attendance list → Mark Present.

If student ID is already present → Do not mark again.

9. Handling Temporary Disappearance

The tracker maintains the student's tracking ID for a predefined number of frames when the face is temporarily blocked or not visible. When the face becomes visible again, the identity can be associated with the existing tracking ID.

10. Justification

Face recognition provides student identity, while tracking prevents repeated recognition of the same person in consecutive frames. Multiple-frame confirmation reduces false recognition. Maintaining an attendance set prevents duplicate entries. Frame resizing and selective face processing reduce computational delay.

5. Smart Home Surveillance and Intrusion Detection

1. Problem Statement

A home security system continuously monitors an entrance using a fixed camera. It must detect significant movement, track moving objects, identify suspicious activity, and generate an alert when an unauthorized person enters a predefined region.

2. Objective

To develop a lightweight OpenCV surveillance system that detects meaningful motion and minimizes false alarms caused by noise, shadows, illumination changes, and dynamic backgrounds.

3. Constraints

- Continuous video monitoring.
- Limited computational resources.
- Camera noise.
- Shadows.
- Changing illumination.
- Moving trees or other background objects.
- Small unwanted movements.
- Real-time alert generation.

4. Input

- Fixed CCTV/webcam video stream.
- Predefined restricted region.
- Motion detection parameters.
- Optional authorized-person database.

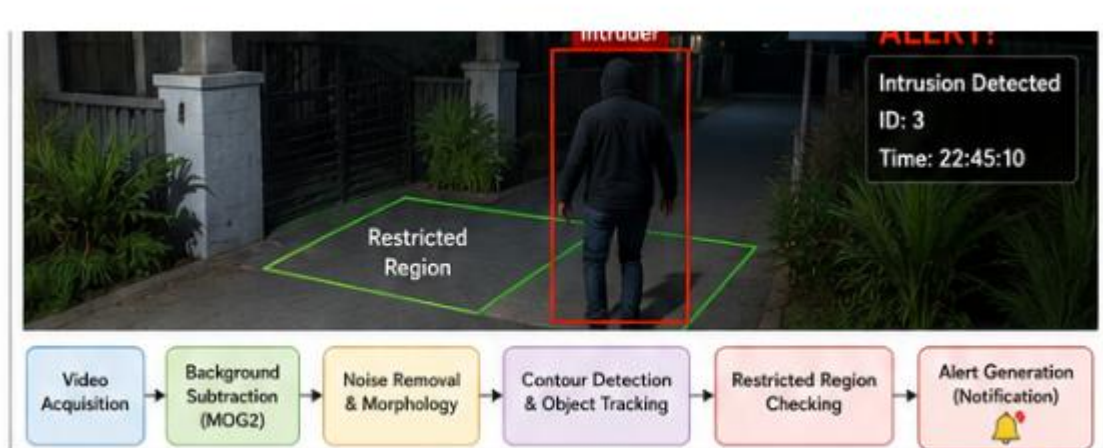
5. Processing / Algorithm

1. Capture video frames using OpenCV.
2. Resize the frames for efficient processing.
3. Apply background subtraction using MOG2 or KNN.
4. Generate a foreground mask.
5. Apply Gaussian/median filtering to remove noise.

6. Apply morphological opening to remove small objects.
7. Apply morphological closing to fill gaps in detected objects.
8. Find contours in the foreground mask.
9. Remove contours below a minimum area threshold.
10. Calculate bounding boxes and centroids of moving objects.
11. Track detected objects across consecutive frames.
12. Define a restricted region at the entrance.
13. Check whether a tracked object enters the restricted region.
14. Analyze movement duration and direction to distinguish meaningful activity.
15. If an unauthorized person enters the region, generate an alert.
16. Display the video, bounding boxes, tracking IDs, and alert status.

6. OpenCV Pipeline

Figure: OpenCV-based system diagram



Camera → Frame Acquisition → Background Subtraction → Filtering → Morphological Operations → Contour Detection → Object Detection → Object Tracking → Restricted Region Check → Intrusion Detection → Alert

7. Output

- Motion detection result.
- Moving-object bounding boxes.
- Tracking IDs.

- Intrusion status.
- Real-time security alert.

8. Reducing False Alarms

- Camera noise: Apply Gaussian/median filtering.
- Small background changes: Ignore contours below a minimum area.
- Shadows: Use shadow detection and color/intensity analysis.
- Illumination changes: Use adaptive background modeling.
- Trees/moving objects: Apply ROI restrictions and persistence checks.
- Temporary movement: Require movement to remain for several frames before generating an alert.

9. Justification

Background subtraction efficiently identifies moving regions in a fixed-camera environment. Filtering and morphological operations remove noise and improve object shapes. Contour area filtering prevents small movements from triggering alarms. Tracking provides information about object movement over time. A predefined restricted region ensures that alerts are generated only when meaningful movement occurs.